

RCROC: TMR Instrumentation and Beam Validation of Observable SEU Response in a RISC-V Soft-Core

L. Nogueira¹, D. Alencar¹, A. Barros¹, A. Pamplona¹, D. Santos², L. Dilillo², J. Silveira¹, M. Andjelkovic³, and F. Vargas³

Abstract—RCROC instruments a RISC-V soft-core with injection-validated, module-level TMR plus configuration-memory scrubbing, latchup recovery, and ECC. Under 70 MeV to 184 MeV protons it detects and localizes per-module upsets online, with alarm activity rising with energy.

Index Terms—SEU, TMR, FPGA, RISC-V, soft-core, proton beam, fault injection, observability.

I. INTRODUCTION

Electronic systems deployed in space, avionics, and other harsh environments operate under a continuous flux of ionizing particles that deposit charge in sensitive circuit nodes and induce single-event effects. SRAM-based FPGAs are widely used in such payloads for their reconfigurability and performance, but the same particles flip configuration- and user-memory bits, producing single-event upsets (SEUs) that can corrupt the implemented logic [1], [2]. RISC-V soft-cores are increasingly adopted as embedded controllers in this setting [3], which makes their resilience a practical concern. Mitigation is addressed at several layers: error-correcting codes (ECC) protect on-chip memories, configuration-memory scrubbing repairs the bitstream, and triple modular redundancy (TMR) masks faults in user logic.

Mitigation alone, however, does not establish that a design is effectively hardened: the protection should also be *observable*, telling which part of the circuit was disturbed, and it must be validated against the real environment, which proton and neutron beam campaigns reproduce under realistic flux and energy.

Prior work hardens RISC-V and other soft processors with TMR and evaluates the result in two complementary ways: *fault injection*, which emulates upsets under tool control, and *irradiation*, which exposes the device to a real particle beam [4]–[6]. In these studies the injection acts on the FPGA configuration memory through an external controller, and, as their authors note, it cannot reach upsets in the design's own

flip-flops [4]. Both evaluations are usually summarized as a single, design-level figure of merit, with offline injection run *after* the campaign to attribute failures that cannot be traced directly. What none of these provide is *online, per-module* observability: a signal, emitted during execution, that identifies which functional unit was disturbed, and when, while the fault is corrected in place. RCROC adds exactly this, reusing the same observability path for a controlled, pre-beam injection campaign, and demonstrates it in a 70 MeV to 184 MeV proton beam (prior soft-core campaigns used neutron or heavy-ion sources).

We target CROC [8], a modifiable open RV32IMC soft-core [9], and extend it into an instrumented, beam-tested variant that we call RCROC (robust CROC), adding module-level TMR backed by configuration-memory scrubbing through the Xilinx Soft Error Mitigation (SEM) IP [10], latchup recovery, and ECC. Our contributions are:

- 1) a fault-tolerant architecture that masks and corrects SEUs while *localizing* them, per module and in time, through TMR sensors with synchronous, in-cycle correction;
- 2) a fault-injection mechanism built into the design itself, at the protected flip-flops rather than in the configuration memory, that drives controlled single- and double-copy faults per module and verifies the expected behaviour deterministically before a campaign;
- 3) a proton-beam campaign at 70 MeV to 184 MeV that validates the architecture under real irradiation.

II. ARCHITECTURE AND INSTRUMENTATION

RCROC is built on CROC [8], an open and extensible RISC-V platform whose processor is the CVE2 (CV32E20) core, a compact two-stage RV32IMC implementation [9]. We adapt CROC to host a per-register *TMR sensor* implemented on the FPGA fabric (Fig. 1). Each protected register is held in three redundant copies feeding a majority *voter*; the voter output drives the downstream logic, so a single faulty copy is outvoted and masked. When any copy disagrees with the majority, a per-module alarm is raised and a synchronous correction rewrites all three copies with the majority value on the next clock edge: the internal state is repaired in one cycle, with no asynchronous feedback. Two faults in the same voting group instead form a wrong majority that propagates, the fundamental 2-of-3 limit of any TMR scheme, which we later exercise directly by injection (Section III). Each TMR cell also embeds a small hardware fault injector at the flip-flop inputs, described in Section III, so that the same cell that protects the state can also be tested through it.

¹Universidade Federal do Ceará (UFC), Fortaleza, Brazil; ²Université de Montpellier, Montpellier, France; ³IHP – Leibniz Institute for High Performance Microelectronics, Frankfurt (Oder), Germany.

Manuscript prepared for RADECS 2026 summary submission. This work has received funding from the European Union's 2020 research and innovation programme under grant agreement No 101008126, corresponding to the RADNEXT project. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Health and Digital Executive Agency (granting authority). Neither the European Union nor the granting authority can be held responsible for them.

The authors thank the AGOR-FIRM/PARTREC irradiation facility (Groningen) for beam time and support during the campaign.

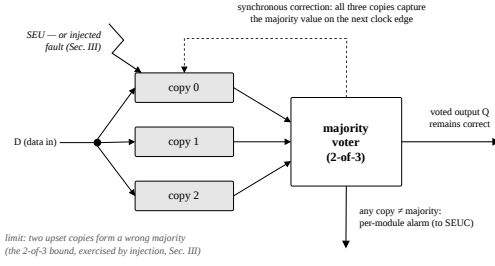


Fig. 1. TMR cell, one per protected flip-flop bit. A single upset copy, real or injected (Section III), is outvoted, raises the per-module alarm, and all three copies are rewritten on the next clock edge; two upset copies defeat the 2-of-3 vote.

Applying the protection takes three steps: the RTL of the core is parsed automatically, the flip-flops of each target module are located, and every plain flip-flop is replaced by a TMR cell. The few modules that the automated pass cannot fully process were converted directly in the source code. The selection targets the sequential state whose corruption persists across cycles and propagates into the architectural behaviour of the core: the fetch path (program counter and fetched instruction), the decode and control state machines that sequence execution, the register file and control/status registers that hold architectural data and trap/privilege state, and the bus-interface response state, whereas purely combinational datapath nodes are re-evaluated every cycle and are naturally masked. The result covers nine CVE2 modules and 1,537 triplicated flip-flop bits, broken down per module in Table I. Since every triplicated bit is an independent target for the beam, the largest modules are expected to concentrate most of the observed upsets; the campaign confirms this expectation (Section VI).

These cells are integrated at the system level through an on-chip *SEU Controller* (SEUC; Fig. 2). Localization is built into the wiring: each protected module drives one bit of an alarm word, so when a fault is detected anywhere in the module its bit is asserted and the disturbed module is named directly. For every module the SEUC also keeps a correction counter and a latched alarm status, and streams them over a UART telemetry channel: the host sees, while it happens, which modules raised an alarm and that their corrections were applied. The same peripheral drives the fault injectors, so injection, detection, correction and reporting all pass through one block and the complete chain can be exercised deterministically before any beam time.

Three complementary protection layers sit outside the CPU TMR and act independently of it. The Xilinx SEM controller continuously scans the FPGA configuration memory and repairs upset frames, covering the layer that TMR does not reach: the description of the implemented circuit itself. Each on-chip SRAM bank is wrapped with a single-error-correcting, double-error-detecting (SECDED) code: a flipped bit in a stored word is corrected on access, and the event is reported through the same telemetry path. Latchup is handled at board level: a supervisor microcontroller monitors the supply current and power-cycles the board when a latchup signature appears (Section IV).

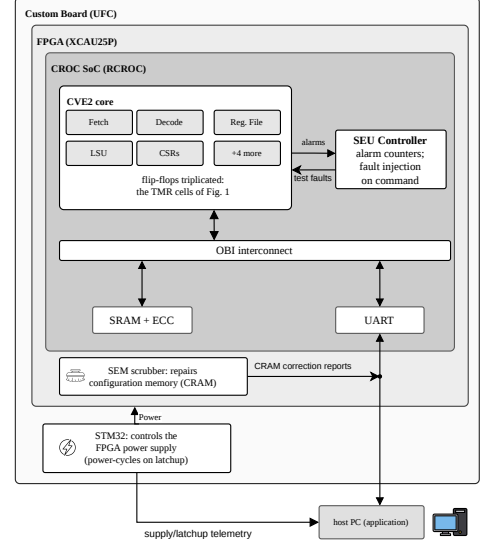


Fig. 2. RCROC system overview: TMR cells (Fig. 1) in nine CVE2 modules, the SEU Controller for alarms and commanded injection, SEM configuration-memory scrubbing, SECDED SRAM, and the STM32 latchup supervisor; each failure class reports to the host on a dedicated, labelled channel.

TABLE I
TMR-PROTECTED CVE2 MODULES AND THEIR TRIPlicated FLIP-FLOP BIT COUNTS.

Module (function)	Protected state	FF bits
Instruction Fetch	PC, fetch address/data	118
Prefetch Buffer	fetch FSM, address	70
Fetch FIFO	buffered instruction, FSM	133
Load/Store Unit	address, data, FSM	38
Controller	control FSM, exception flags	61
Control/Status Registers	CSRs (trap/privilege state)	46
Register File	$\times 0 - \times 31$	992
Instruction Decode	decode FSM, branch, immediate	71
Bus Interface (OBI)	response state	8
Total triplicated flip-flop bits		1,537

III. PRE-BEAM FAULT-INJECTION VALIDATION

Before the beam we verified the complete detect–correct–report chain by deterministic fault injection. Unlike the configuration-memory injection of prior soft-core studies [4]–[6], the injector here is part of the design itself: every protected flip-flop carries a small hardware injector at its inputs, wired to the SEUC. On command from the host, software running on the core under test writes the SEUC injection registers: a per-module mask selects which modules will receive the injected fault, and a second field selects which of the three copies to disturb. The SEUC then asserts a single-cycle pulse that inverts the input of the selected copy, and on the next clock edge that copy’s flip-flop captures the inverted value. The upset is at that point physically held in the user state, exactly the final condition a real SEU would leave in that flip-flop. Pulses are issued at a fixed cadence of one every 100 ms ($\approx 2 \times 10^6$ clock cycles at 20 MHz), so the protected path is observed continuously across tens of millions of cycles.

The expected response follows the voter cycle by cycle. In the cycle after the pulse, the flipped copy disagrees with the majority: the per-module alarm rises, while the other two copies keep the voted output, and therefore the architectural behaviour, correct. On the following edge the synchronous correction rewrites all three copies with the majority value, and the alarm falls. A *double-copy* fault in the same voting group makes the faulty pair the majority instead, so the corrupted value propagates and the core diverges: the designed failure mode of any 2-of-3 scheme.

The observed behaviour matched this prediction for every injected fault. We swept the five modules on which a single-copy fault maps to exactly one detection and one correction per pulse (load/store unit, controller, control/status registers, register file, and instruction decode), spanning the protected-state range from the small control FSMs to the 992-bit register file. In every module the detection and correction counters advanced in lockstep with the injected pulses, one for one, and a dedicated repair counter confirmed that every injected alarm lasted exactly one cycle, one repair per fault. A double-copy fault on the same path drove the core into the predicted failure on the first pulse.

Two control runs confirmed that the counters respond only to real faults: the same pulse sequence with the module-selection mask cleared, so that no copy is flipped, left every counter at zero, as did running the system with no pulses at all.

The four remaining modules carry the same TMR cells and raise the same per-module alarms; what differs is their recovery path, which is why a one-to-one per-pulse count does not apply to them. A fault in the fetch path invalidates instructions already in flight, so the safe recovery is to flush the pipeline and refetch rather than to repair the state in place; the bus interface, with eight bits of response state, relies on error detection. We injected faults into these modules as well: every fault was detected and attributed to its module, and the fetch path recovered through the flush each time. Their behaviour under irradiation is reported in Section V.

Two aspects bound this campaign's scope. First, injection is applied module by module: a whole-module pulse on the 992-bit register file would assert an unphysically large number of simultaneous upsets, whereas a per-module policy keeps the emulated event close to a single particle strike. Second, this is a *deterministic* verification of the correction mechanism, not a statistical estimate of SEU sensitivity as in configuration-memory campaigns that sample millions of random bits [4], [5]: every fault had a predicted outcome, and the observed outcome matched it on both sides of the 2-of-3 boundary. These controlled faults establish, before any irradiation, that the architecture detects, corrects and reports single-copy upsets and fails only where any TMR scheme must.

IV. EXPERIMENTAL SETUP

Having established this on the bench, we characterized the same design under real irradiation. The campaign ran at the AGOR cyclotron irradiation facility (AGOR-FIRM, PARTREC, Groningen) [11]. The device under test was an AMD/Xilinx Artix UltraScale+ FPGA (XCAU25P-1FFVB676I, 16 nm

FinFET) on a custom board developed at UFC, hosting the soft-core at 20 MHz running a continuous compute workload. The proton beam was collimated onto the FPGA die alone, so that only the soft-core and its configuration memory were irradiated and not the surrounding board electronics. Three energies (70, 120, 184 MeV) were obtained by degrading the AGOR primary beam with a multi-leaf aluminium degrader (energy verified by a range telescope). Proton flux (5×10^6 p/cm²/s to 1×10^7 p/cm²/s) and accumulated fluence were monitored for each exposure by the facility's calibrated, non-interceptive ionization chambers referenced to a detector at the DUT and logged by the AGOR-FIRM control system.

The test system comprises four instruments around the DUT (Fig. 2); the first three each own one failure class, and the fourth records them all. First, the SEUC streams the per-module TMR alarms and correction counters over UART, covering upsets in the protected core state. Second, the Xilinx SEM controller independently reports every configuration-memory upset it repairs, and flags the rare patterns it cannot correct. Third, an STM32 supervisor monitors the board supply current, power-cycles the board when a latchup signature is detected, and reports each trip and reboot over its own telemetry channel. Fourth, a host application timestamps and logs everything for the whole campaign, over two serial links (Fig. 2). This organization makes failure attribution direct: a supply event arrives labelled as a latchup trip, a configuration upset arrives labelled by the scrubber, and a disturbance of the protected state arrives labelled with the module that raised it. A residual anomaly that arrives on none of these channels is bounded by elimination, rather than left unexplained.

V. RESULTS

A. Per-module detection and localization under beam

Under irradiation the SEUC observability path was active: it flagged and *localized* disturbances to individual modules during execution, and in most exposures the core kept executing. A disturbance is recorded as an *alarm episode*, a contiguous period during which a module's voter reports disagreement.

The alarms attributable to the beam concentrate in the largest state elements: the 992-bit register file dominates, followed by the fetch FIFO, while the smaller modules recorded none, matching the expectation set in Section II and developed in Section VI. This per-module localization, which shows *where* the beam disturbed the core while the disturbances remained correctable, is precisely what an aggregate, device-level figure of merit cannot express.

When an exposure did go down, the telemetry separated the cause (a supply latchup handled by the board supervisor, a configuration upset outstanding until SEM scrubbing, or a CPU-level failure) instead of leaving it unexplained. Taken together, this live localization and cause attribution are the per-module, online observability that the architecture was designed to provide.

B. Disturbance activity across proton energy

To show how the disturbance registered by the instrumentation tracks proton energy, we compare three exposures, one at

each of 70, 120 and 184 MeV, that accumulated the same total proton fluence (1×10^{10} p/cm², delivered at 5×10^6 p/cm²/s). Because the three exposures received the same number of protons per unit area, the energy is the only variable that changes between them, and a difference in the response is attributable to it.

The beam calls for a different measure than the bench. Under controlled injection each fault is a discrete event in a protected flip-flop, repaired within one clock cycle. Under the beam a disagreement can instead be *held*: a configuration-memory upset in the TMR cell itself, in its voter or routing, re-creates the disagreement on every clock edge, re-triggering the synchronous correction until scrubbing or reconfiguration restores the fabric [4], [6]. During the longest alarm episode the detection and correction counters, strictly one-for-one under injection (Section III), diverged, the signature of one held disagreement corrected repeatedly rather than of discrete upsets. Raw counts therefore do not enumerate events under beam, and we quantify the disturbance by duration: how long a module's alarm stayed asserted, as a fraction of the exposure, and the number of distinct episodes. On this measure the response concentrated exactly where the architecture has the most exposed state: only the register file, the largest protected module with 992 of the 1,537 triplicated bits, showed beam-attributable activity at any of the three energies. Its alarm was asserted for none of the exposure at 70 MeV, 10 % of it at 120 MeV, and 55 % at 184 MeV, in a single episode each time. The disturbance is thus longer-lasting on the same dominant module as energy rises; what this enables is developed in Section VI.

Independently of the TMR, the SEM scrubber's configuration-memory correction rate per unit fluence rose with energy across the same exposures ($\approx 4.5, 8.0, 12.4 \times 10^{-10}$ cm²): a device-level confirmation that the rising alarm activity tracks a real environmental gradient.

VI. DISCUSSION

Two aspects of the beam results deserve emphasis. The first is that the observed distribution of alarms is physically consistent. Each triplicated bit is an independent target for the beam, so under a uniform field the probability that a module registers an upset should grow with the amount of state it holds. That is what the campaign recorded: the 992-bit register file dominated the alarms, the mid-sized fetch FIFO followed, and the small state machines stayed silent. The localization therefore behaves like area-proportional upset physics rather than a telemetry artifact. It also turns Table I into an actionable ranking of where additional hardening effort would pay first.

The second is what the energy comparison says about the usefulness of the read-out. At matched fluence, the same dominant module stayed in alarm for longer as the proton energy rose, from no alarm at 70 MeV to more than half of the exposure at 184 MeV. The architecture thus inferred, from its own protected state and while correcting it, that the environment had become harsher: a longer-held alarm on the same module accompanied the higher energy. Within the limits of a single exposure per energy, this is the contribution in deployment

terms: the instrumentation names the module under stress and indicates when the stress intensifies, instead of reporting a single device-level number after the fact. On-chip, module-resolved radiation monitors of exactly this kind are the input that self-adaptive mitigation schemes rely on to decide when to raise or relax protection [7].

VII. CONCLUSION

RCROC set out to do three things, and the campaign closes each. First, the architecture localizes upsets per module, in time, while correcting them: in the pre-beam injection every controlled fault in all five swept modules was detected and corrected within a single cycle, and under 70 MeV to 184 MeV protons the same path localized real disturbances to individual modules as they happened. Second, the design-integrated injection mechanism reproduced on demand both the single-copy regime and the double-copy failure regime, pre-validating the design before any beam time. Third, the beam campaign exercised the protected core under real irradiation, where the alarm activity grew monotonically with energy at matched fluence and the telemetry attributed every system-down to its cause. Building on this, future work will sharpen the localization from the functional module down to the physical TMR cell on the fabric, and will gather repeated exposures per energy to turn this illustrative trend into a quantitative one.

REFERENCES

- [1] R. C. Baumann, "Radiation-induced soft errors in advanced semiconductor technologies," *IEEE Trans. Device Mater. Rel.*, vol. 5, no. 3, pp. 305–316, 2005.
- [2] J. Benfica *et al.*, "Analysis of SRAM-based FPGA SEU sensitivity to combined EMI and TID-imprinted effects," *IEEE Trans. Nucl. Sci.*, vol. 63, no. 2, pp. 1294–1300, Apr. 2016.
- [3] S. Di Mascio, A. Menicucci, E. Gill, G. Furano, and C. Monteleone, "Leveraging the openness and modularity of RISC-V in space," *J. Aerosp. Inf. Syst.*, vol. 16, no. 11, pp. 454–472, 2019.
- [4] A. E. Wilson, S. Larsen, C. Wilson, C. Thurlow, and M. Wirthlin, "Neutron radiation testing of a TMR VexRiscv soft processor on SRAM-based FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 68, no. 5, pp. 1054–1060, 2021.
- [5] A. E. Wilson, M. J. Wirthlin, and N. Baker, "Neutron radiation testing of RISC-V TMR soft processors on SRAM-based FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 70, no. 3, pp. 603–610, 2023.
- [6] M. Wirthlin, A. Keller, C. McCloskey, P. Ridd, D. S. Lee, and J. Draper, "SEU mitigation and validation of the LEON3 soft processor using triple modular redundancy for space processing," in *Proc. ACM/SIGDA Int. Symp. FPGAs*, 2016, pp. 205–214.
- [7] J. Chen, T. Lange, M. Andjelkovic, A. Simevski, L. Lu, and M. Krstic, "Solar particle event and single event upset prediction from SRAM-based monitor and supervised machine learning," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 2, pp. 564–580, 2022.
- [8] M. Sauter *et al.*, "Croc: An end-to-end open-source extensible RISC-V MCU platform to democratize silicon," arXiv:2502.05090, 2025.
- [9] OpenHW Group, "CORE-V CVE2 (CV32E20): A 32-bit RISC-V CPU core," 2023. [Online]. Available: <https://github.com/openhwgroup/cve2>
- [10] AMD/Xilinx, "UltraScale architecture soft error mitigation controller v3.1," LogiCORE IP Product Guide PG187, 2022.
- [11] S. Brandenburg, R. Ostendorf, M. Hofstee, H. Kiewiet, and H. Beijers, "The irradiation facility at the AGOR cyclotron," *Nucl. Instrum. Methods Phys. Res. B*, vol. 261, no. 1–2, pp. 82–85, 2007.